Lesson notes · for you

# ETL with Python and SQL

Data engineering · Level 4 · Lesson 2

**Time**
17 min

**Materials**
Your phone or a computer, for the Python and SQL steps; Paper, to sketch a pipeline

## Learning goals

- Build extract, transform, load.
- Make loads idempotent: delete-then-insert, or upsert.
- Load incrementally and log every run.

## Activity: Break the pipeline

1. Draw (4 min): Sketch the school attendance pipeline: sources, transform, district table, schedule.
2. Fail (5 min): For each step, write what's left behind if it fails halfway, and what a plain rerun would do.
3. Fix (5 min): Rewrite the load as delete-then-insert or upsert, and decide where the watermark is saved.
4. Log (3 min): List what the run log needs so you'd spot last night's failure in a minute.

**The lab's numbers (84 of 90, and 152 after a bad retry) make a good worked example.**

## Discussion

- Why must the watermark be saved after the load?
- When would you choose upsert over delete-then-insert?
- Which failures should never be retried?

## Check yourself

- Write the two SQL statements of an idempotent daily load.

**Try it this week**
Think of a job you repeat, like backing up your phone's photos. What happens if it runs twice, or stops halfway? Redesign it so a rerun is always safe.

Worksheet

# ETL with Python and SQL

Data engineering · Level 4 · Lesson 2

Name _____ Class _____ Date _____

**1** **Which load is idempotent?**

- ◯ Insert, then remove duplicates by hand next week
- ◯ Delete the day's rows, then insert them
- ◯ Insert the day's rows on every run

**2** **Put the steps of a nightly incremental load in order.**

- ☐ Extract only the rows newer than it
- ☐ Read the watermark: the last time loaded
- ☐ Save the new watermark
- ☐ Transform: clean, type and count
- ☐ Load: delete the day's rows, then insert

**3** **The transform step. register.csv was typed by hand: some statuses are " p" or "P ", and one is "?". Clean each status with strip() and upper(), count the P's per class, set aside anything that isn't P or A, and print: 5A 4, 5B 3, 6A 5, then aside 1.**

```
import csv
present, aside = {}, 0
with open("register.csv") as f:
    for row in csv.DictReader(f):
        s = row["status"]
        if s == "P":
            c = row["class"]
            present[c] = present.get(c, 0) + 1
for c in sorted(present):
    print(c, present[c])
print("aside", aside)
```

Write your program:

**4** **Last night's load failed after writing two of Monday's classes into daily. Make the load idempotent: before inserting today's rows, delete the Monday rows already in daily. The last line shows the totals per day: Fri 83 and Mon 84.**

```
INSERT INTO daily
SELECT * FROM today;
SELECT day, SUM(present) FROM daily
GROUP BY day ORDER BY day;
```

Write your program:

**5** **Put the lines of an incremental load in order, so a failed load never loses rows.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

- [ ] `if new:`
- [ ] `load(new)`
- [ ] `save_watermark(max(r["time"] for r in new))`
- [ ] `watermark = read_watermark()`
- [ ] `new = rows`
- [ ] `new = [r for r in rows if r["time"] > watermark]`

✎ **Think of a job you or your family repeat on a schedule (a backup, a bill, a shopping list). What happens if it runs twice, or stops halfway?**

Check your answers

# ETL with Python and SQL

Data engineering · Level 4 · Lesson 2

**1** **Delete the day's rows, then insert them**

Clearing the day first means one run or five give the same table.

**2** **Top to bottom: 2, 1, 5, 3, 4**

The watermark decides what to extract, so it comes first; it's saved only after the load succeeds, so a failed run is simply repeated, and the delete-then-insert makes that repeat safe.

**3**

```
import csv
present, aside = {}, 0
with open("register.csv") as f:
    for row in csv.DictReader(f):
        s = row["status"].strip().upper()
        if s not in ("P", "A"):
            aside += 1
        elif s == "P":
            c = row["class"]
            present[c] = present.get(c, 0) + 1
for c in sorted(present):
    print(c, present[c])
print("aside", aside)
```

Cleaning first turns " p", "p" and "P " into P, so 5A has 4 present and 5B 3.

**4**

```
DELETE FROM daily
WHERE day IN (SELECT day FROM today);
INSERT INTO daily
SELECT * FROM today;
SELECT day, SUM(present) FROM daily
GROUP BY day ORDER BY day;
```

Deleting the day's rows first clears what the failed run left, so the insert writes Monday exactly once: 84.

**5** **Top to bottom: 3, 4, 5, 1, X, 2**

```
watermark = read_watermark()
new = [r for r in rows if r["time"] > watermark]
if new:
    load(new)
    save_watermark(max(r["time"] for r in new))
```

Read the watermark, keep only the newer rows and, if there are any, load them and save the new watermark last.